MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Arrays

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning outcomes

- Review arrays declaration in Python

- Understand
  - How arrays are stored in memory
  - How elements are accessed
  - How to extend to multidimensional arrays
  - Why bounds checking is needed

Uses of Arrays

- Arrays are heavily used in scientific computation:
  - Iterative solutions to systems of partial differential equations;
  - Finite Difference and Finite Element Methods
  - Matrix operations in coupled systems…
  - E.g. weather forecasting, motor design, aerodynamics…
- Computations are long & time critical.
- Efficient array access is vital.
- Fortran was designed for this, C is as efficient, Java, Python etc a bit less so (but not enough to matter in most cases).
- In most situations execution efficiency is not the greatest concern, hence the widespread use of Java and Python.

Arrays in Python

- The simplest form of array in Python is an array of primitive types:
  - a_int = array(‘i’,[0,0,0,0,0,0,0,0,0,0])
  - a_chr = array(‘u’,’one’)
- This is similar to the string examples used in lectures and labs.
- How is the array used, and how is it stored in memory -where are the elements of the array?

How do we use arrays

- Could be in a simple loop:

```
for i in range(0,10):
   a[i] = i
```

- Or more complex

```
for i in range(1,10,2):
   a[i] = i
;or
x = a[ b * c + d ]
```

- Or multi-dimensional:

```
for i in range(1,10):
   for j in range(1,10):
       a[i][j] = 0
```

Arrangements of arrays in memory

- The array is stored in memory
- Each element of an integer array is 32 bits (4 bytes)
- In simple 1-D arrays the elements are stored consecutively
- How do we find an individual element?
- We only need to know the **base address** of the array (e.g. 1FAB400) and the element number (or "**index**")
- element i is at base address + 4i, e.g a(2) is at 1FAB400 + 8 = 1FAB408

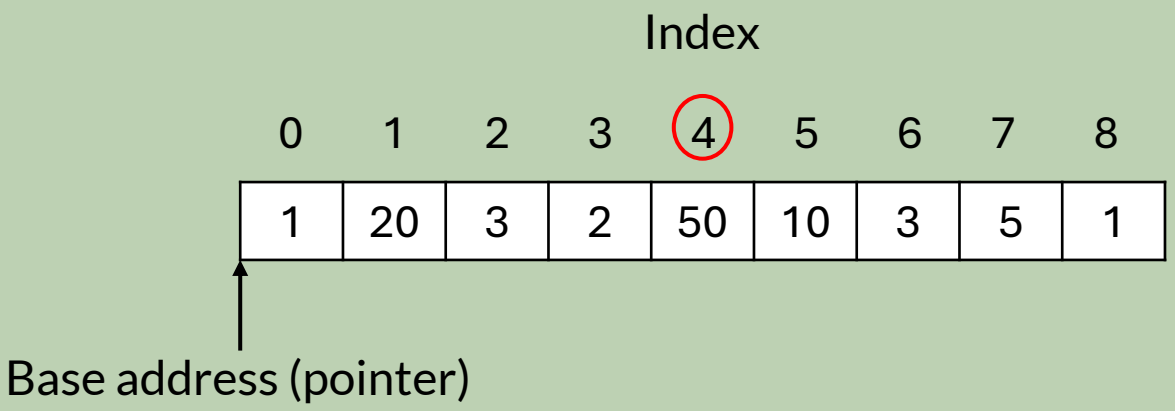
1FAB4000	a(0)
1FAB4004	a(1)
1FAB4008	a(2)
1FAB4024	a(9)

How to access arrays

- Element i is at base address + 4i
- The processor has to store the base address for each array
- The location or register that holds the base address is sometimes called the **pointer** to the array because it shows where the array is stored.
- The processor also needs to know which element of the array to access – it needs an **offset** or **index**, i.
- To load a[i] the processor has to:
  - Find the base address of the array a (from the pointer to the array)
  - Calculate the address of the required element i as base address + 4*i
  - Load the 4 byte word from that address

How to access arrays

- Ele
- Th
- Th
  arr
- Th
  or
- To
  -
  - C
  - Load the 4 byte word from that address



Using an array in RISC-V

- **Declare the array:**

```
; Array of student heights
; Note: single byte per student: +ve integer heights <256cm
; Note: the program does not have a zeroth student
; so pad the array
heights defb 0,150,160,165,155,150,170,175,180,155,160
```

- **Set up base address and index:**

```
li a0, 10       ;Index set to student number 10
la a1, heights ;Make a1 the pointer to the array
```

- **Access element 10 of the array:**

```
add a1, a1, a0 ;Set the current student number to 10
lb  a2, 0[a1]  ;Height is at base address + I
```

A complete program

```
      li   a0, 0        ;Set the total height to zero
      li   a1, 0        ;Set the current student height to zero
      li   a2, 10       ;Set the current student number to 10
      li   a3, 2000     ;Set length of the lecture theatre to 2000cm
      la   a4, heights  ;Make a4 the pointer to the height array
      beqz a2, end
      addi a2, a2, a4
loop  lb   a1, 0[a2]    ;Height is at base address + i
      add  a0, a0, a1   ;Accumulate the total height
      subi a2, a2, 1    ;Next student number is...
      bgt  a2, a4, loop ;If next student number !=0 do it again
end   slt  a5, a3, a0   ;If total height > length then set result to 1
stop  li a7, 5
      ecall

; Array of student heights
heights  defb 0,150,160,165,155,150,170,175,180,155,160
```

Ways to access arrays

- Accessing an array consecutively:

```
for i in range(1,10):
    a[i] = 0
```

- For an array of bytes this might be a loop containing:

```
lb   a1, 0[a4] ; fetch [a4]
addi a4, a4, 1 ; then a4⇐a4+1
```

- If the array is 32 bit (4 byte) integers:

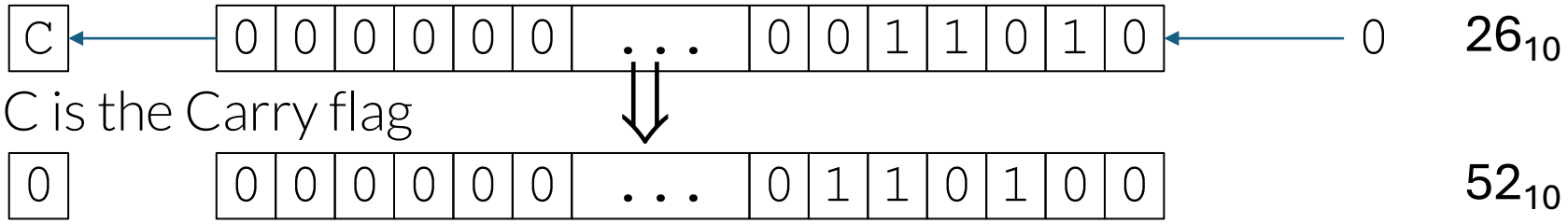
```
lw   a1, 0[a4]  ; fetch [a4]
addi a4, a4, 4 ; then a4⇐a4+4
```

- We probably prefer to leave the pointer (base address) unchanged and so use an index register for the offset. For an array of bytes:

```
li   a2, 0
add  a2, a2, a4
lb   a1, [a2]
addi a2, a2, 1
```

Multiply/divide by powers of 2

slli – shift left logical immediate - multiply by 2:

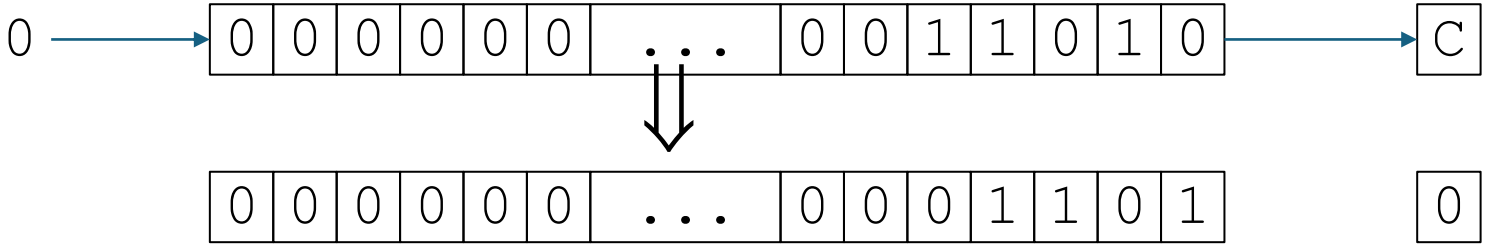


26_{10}

C is the Carry flag

52_{10}

srli – shift right logical immediate - divide by 2:



26_{10}

13_{10}

- We can slli or srli by any number of bits up to 31,
- …so we can multiply/divide by 2^n for integer n=1…32
- The Carry flag enables longer multiplication

Out of bounds accesses

- An attempt to access an array element that is outside the memory area of the actual array, e.g. :

```
for i in range(1,N):
    a[i] = i
```

What if N = 20?
If N is a constant then it might be possible to detect the problem at compile time and issue a warning.

- However:

```
x = a[ b * c + d ];
```

If **b**, **c** and **d** are variables the compiler cannot know the values that `b * c + d` will have at run time.

- What would happen if we tried to load from or store to a location outside the declared range of the array?
- Does this matter?

Arrays bounds checking

- If we had code of the form `b = a[x * x]`
- If a2 is the pointer and contains the base address, a1 is used as the index and a0 contains x:

```
mul  a1, a0, a0
slli a1, a1, 2
add  a1, a1, a2
lw   a3, 0[a1]
```

- To check that a1 is within bounds when the `lw` is executed:

```
mul  a1, a0, a0
li   a4, N      ; compare a4, N
bge  a1, a4, ErrorCode ; if index > N
slli a1, a1, 2
```

- The size of the array has to be stored somewhere – when the compiler creates ASM code for the creation of an array it has to store the array bounds as well as the array itself

Arrays bounds checking

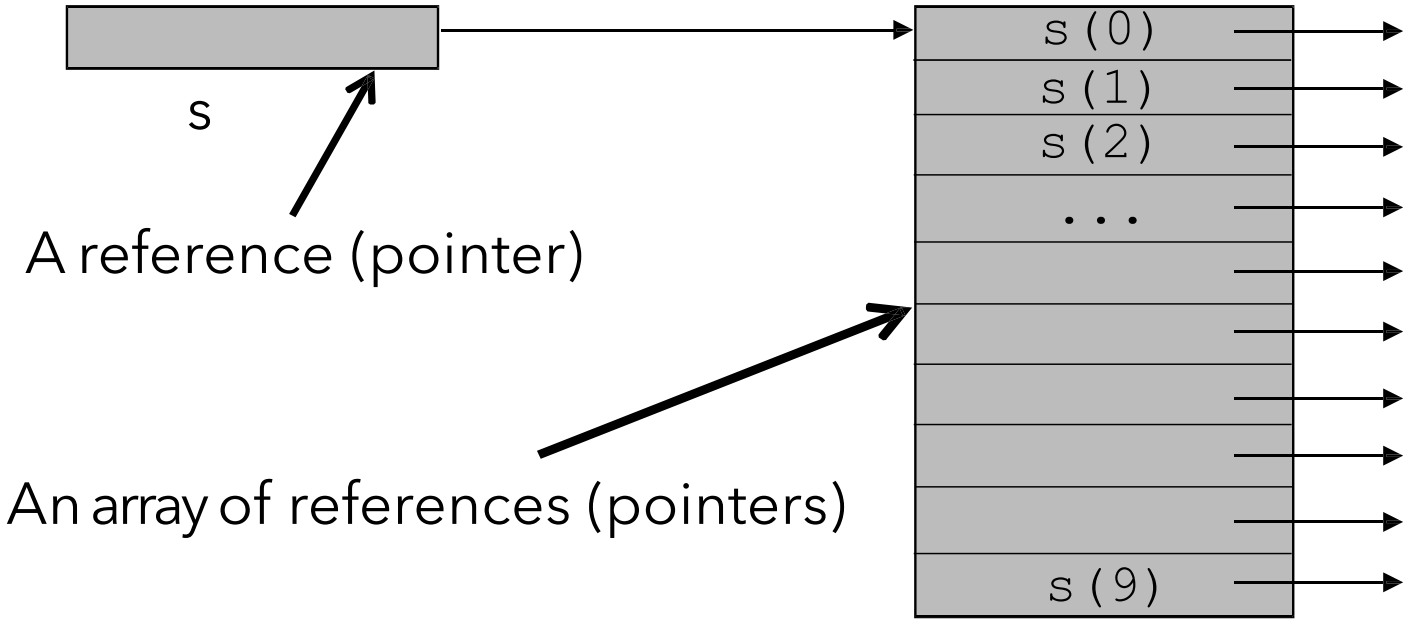
- If In general there could be attempts to access with index<0: the lower bound also has to be checked
- The compiler could cause the target address to be checked against both upper and lower array bounds
- Bounds checking requires additional code and may require additional registers, additional memory accesses etc
- …every time an array is accessed
- Bounds checking takes time and resources
- Some compilers may provide the option of switching bounds checking off – “full optimisation” for speed and code size
- …but only use this if you are sure that the code cannot generate an array index out of range, whatever the data.

OS memory bounds checking

- The OS will have a memory management system:
- Users will have an allocated space
- Users should not be able to access memory outside their allocated space
- It may be that user code space and data space are separately managed by the OS
- That does not prevent user code from accessing an invalid part of the user space so the code still needs to do bounds checking.
- It should not be possible for a user to access memory that is allocated to the OS or to other users
- It should be that attempted out of bounds memory reads/writes can only cause problems for the user and not crash the OS or enable security breaches
- Note that “should” is used repeatedly: it’s hard to write an OS that guarantees all of this.

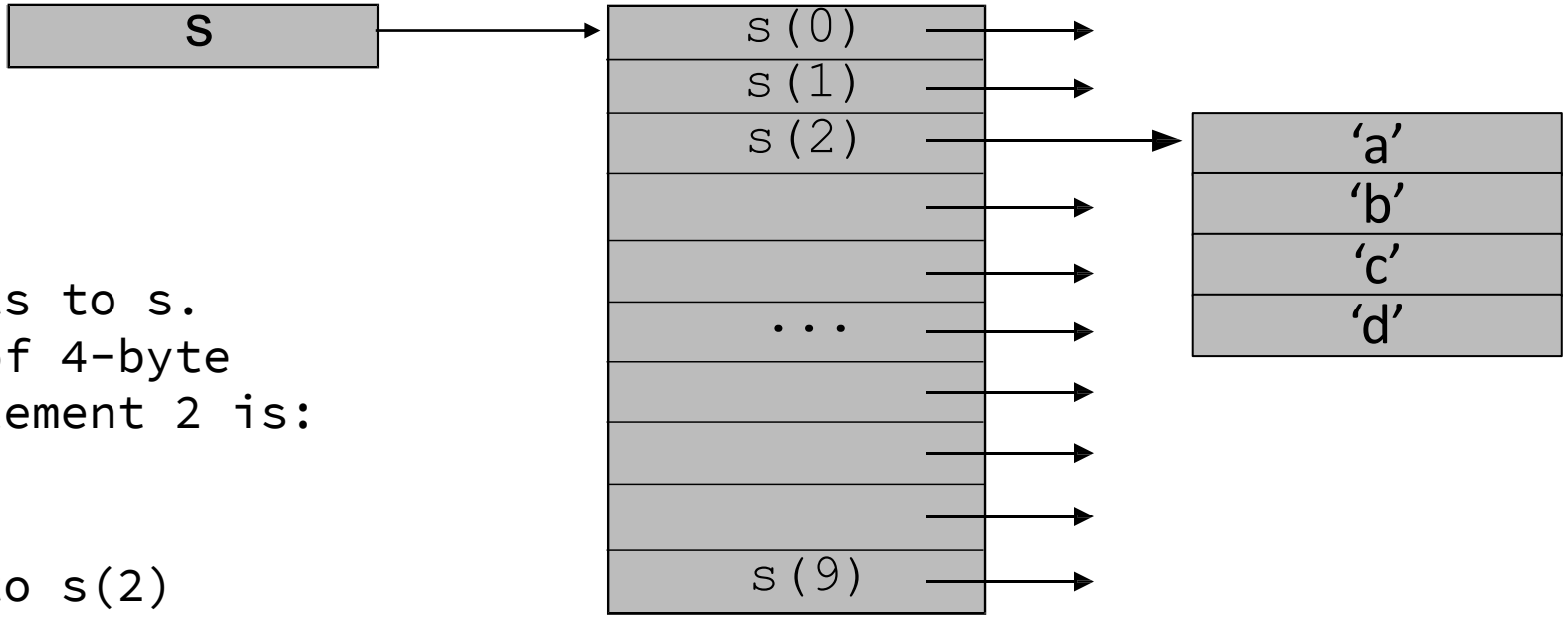
Arrays of objects

- Python arrays can be arrays of objects, not just bytes or integers
- e.g. s = ["str1", "str2", "str3", ...]
- Creates an array of strings…
- …as an array of null references…



Arrays of strings

- ...then create an array of strings utilising 'new' elements
- e.g. s[2] = "abcd"



In RISC-V:

```
;Assume a1 points to s.
;s is an array of 4-byte
;addresses so element 2 is:
addi a1, a1, 8
lw   a0, 0[a1]
;a0 now points to s(2)
lb   a2, 0[a0];
;a2 now contains 'a'
addi a0, a0, 1
lb   a2, 0[a0]
;a2 now contains 'b'
etc
```

Each string in the array s is an object that has its own size

Multidimensional arrays

- Can have multi-dimensional arrays, e.g.

  m = [[0,1,2,3],[4,5,6,7]]
- This could be thought of like a 2-D matrix:

m(0,0)		m(0,2)
m(1,0)		
m(2,0)		m(2,2)

- matrix indices: (row, column)
- How are the elements stored?

Storage of elements

- In some languages the entire array is stored contiguously in memory.
- If the size of the array is (M,N) (in this case 3x3 rows x columns) then:
  - a(i,j) can be accessed as if it were element j+N*i of an 1-D array of size MxN
  - The whole array can be accessed sequentially by autoincrementing:

```
lw   a0, 0[a1]
addi a1, a1, 4
```

  - This example is "row-major" as is used in C
- This can be very fast/computationally efficient.
- Row and column order is different in different languages. Read: https://en.wikipedia.org/wiki/Row-_and_column-major_order

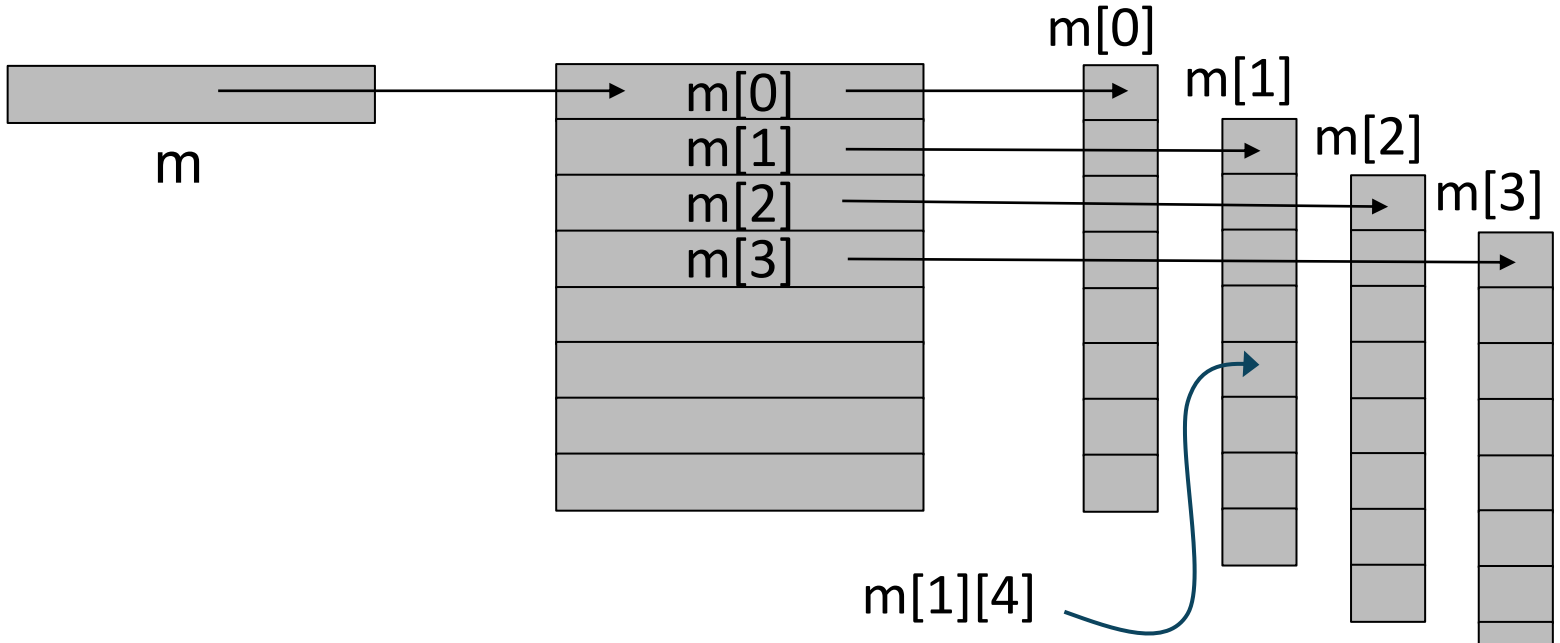
a(0,0)
a(0,1)
a(0,2)
a(1,0)
a(1,1)
a(1,2)
a(2,0)
a(2,1)
a(2,2)

Multidimensional arrays Python

- Python is also an object oriented language
- In Python multi-dimensional arrays are arrays of objects:
    - A 2-D integer array is an array of ‘array of int.’
- Can be extended to any number of dimensions
    - A 3-D array is an array of 2-D arrays etc.
- Logical, but as the number of dimensions increases access becomes increasingly complex/slow.

Python 2-D array

- A 2-D array m[i][j] is an array of 1-D arrays
- m points to an array of pointers, each points to a 1-D array (rows of the 1-D array)
- m[i][j] implies row-major order - can be rewritten (m[i])[j]



- Two memory accesses to access an element of the array
- Less efficient than contiguously stored arrays

Summary

- Arrays are used widely in high level languages
- Different languages assume different underlying memory representations for higher dimensional arrays
- For 2-D arrays either 2 memory accesses or integer multiplication are needed to access specific elements
  - It can matter, check before coding in a new language
- Array bounds checking is slow but necessary to prevent crashes or security breaches.

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Assemblers

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning Outcomes

- Understand how an assembler works
- Understand limitations of assemblers

Simple Assembler - steps

1. Strip out comments

2. Identify named constants, labels etc, e.g.:

```
offer    equ 100
discount subi a0, a0, offer
```

3. Identify instructions

4. Identify data or variable space to be located in memory

5. …and assign an address to each instruction, data word etc

6. Determine the values of all constants, labels etc
   - The value of "discount" is the address of the instruction
   - The value of "offer" is defined by the equ directive

7. Convert text instructions, data values etc into binary

   - e.g. `add a3, a1, a2` → `E0813002`

Simple Assembler

- An assembler has to scan the text version of the program and operate upon it.
- Can it scan the program just once from start to finish and convert line by line to binary?
- In some very simple cases, yes. For example:

```
li a1, 17          →    E3A01011
li a2, 20          →    E3A02014
add a0, a1, a2     →    E0810002
```

- Only for Sequences of instructions/data that can be individually converted to binary/hex form without reference to anything else

Is it really that simple?

- How about this:

```
00000000: E28F4028    ;          la    a4, Data
00000004: E5941000    ;          lw    a1, 0[a4]
00000008: E5942004    ;          lw    a2, 4[a4]
0000000C: E5943008    ;          lw    a3, 8[a4]
00000010: E0810002    ;          add   a0, a1, a2
00000014: BA000001    ;          blt   a0, a3, Lower
00000018: E3A0002B    ;          li    a0, '+'
0000001C: EA000000    ;          j     Print
00000020: E3A0002D    ; Lower    li    a0, '-'
00000024: E3A0002F    ; Print    li    a7, 0
00000028: EF000000    ;          ecall
0000002C: 00000011    ; Data     defw 17, 20, 35
00000030: 00000014    ;
00000034: 00000023    ;
```

- Single pass assembly is only possible if the program is a simple sequence of instructions with:
  - no references to data that is located later in the code
  - no branches forward
  - no position-dependent pseudo instructions (why not?)

A two-pass assembler

- Forward references cannot be resolved in one pass:

```
00000014: BA000001    ;          blt  a0, a3, Lower
00000018: E3A0002B    ;          li   a0, '+'
0000001C: EA000000    ;          j    Print
00000020: E3A0002D    ; Lower    li   a0, '-'
```

- What is "`Lower`"?
- The assembler has to make two passes through the code
- Pass 1:
  - assign an address to each instruction or data
  - record any labels or names encountered with their numerical value in a "Symbol Table"
  - Check that each symbol is defined only once
- Pass 2:
  - replace any labels or names with their values
  - convert each instruction to binary

Example .kmd file

```
KMD
00000000: E28F4028    ;         la   a4, Data
00000004: E5941000    ;         lw   a1, 0[a4]
00000008: E5942004    ;         lw   a2, 4[a4]
0000000C: E5943008    ;         lw   a3, 8[a4]
00000010: E0810002    ;         add  a0, a1, a2
00000014: BA000001    ;         blt  a0, a3, Lower
00000018: E3A0002B    ;         li   a0, '+'
0000001C: EA000000    ;         j    Print
00000020: E3A0002D    ; Lower   li   a0, '-'
00000024: E3A0002F    ; Print   li   a7, 0
00000028: EF000000    ;         ecall
0000002C: 00000011    ; Data    defw 17, 20, 35
00000030: 00000014    ;
00000034: 00000023    ;

Symbol Table: Labels
: Lower                           00000020   Local -- RISC-V
: Print                           00000024   Local -- RISC-V
: Data                            0000002C   Local -- RISC-V
```

Difficulties for a 2-pass assembler

- position-dependent pseudo-instructions:
- e.g. `la`
  - `la` uses PC relative addressing to load an address:
  - `la a0, mystring`
  - → `auipc a0, offset[31:12]`

    `addi  a0, a0, offset[11:0]`
  - the offset is calculated by the assembler and put in the instructions as a literal offset.

Difficulties for a 2-pass assembler

```
        la   a4, Data
...
...
Data  defw 17, 20, 35
```

- How is this assembled?
- The address of Data is put into a4 by adding an immediate offset to the PC to calculate the address of Data
- The assembler calculates the offset during assembly and inserts the value
- Not possible until the 2nd pass: address of Data is unknown in pass 1

Difficulties for a 2-pass assembler

```
        la   a4, Data
...
...
Data  defw 17, 20, 35
```

If the offset is greater than 12 bits, more instructions would be needed, which would change the address of Data, requiring more passes

- How is this assembled?
- The address of Data is put into a4 by adding an immediate offset to the PC to calculate the address of Data
- The assembler calculates the offset during assembly and inserts the value
- Not possible until the 2nd pass: address of Data is unknown in pass 1

A bit more detail

- The process of converting text to binary isn't trivial.
- Can be broken down into 4 phases:
  1. Lexical (word) analysis:
     Convert sequences of characters into sequences of tokens
  2. Syntactic (structural) analysis:
     Identify instructions, check they are valid
  3. Semantic (meaning) analysis:
     Find the values of named entities
  4. Binary code generation:
     Generate final machine code
- Very brief description for a simple assembler follows

Lexical analysis: characters -> words

```
Mask  equ 0x0000000F
start andi a0, a1, Mask; Zero bits except LSN
```

- Scan the line character by character.
- Identify ‘tokens’, ‘lexemes’ or ‘lexical units’ by detecting spaces and other delimiting features in the language (, ;)

start	andi	a0	,	a1	,	Mask	;	comment

Syntactical analysis

- What are the components of a line of RISC-V assembly language program (text)?
  - labels
  - operation code (mnemonics)
  - operands (registers)
  - constants/literals
  - addresses
  - comments
  - delimiters (, ;)

- May be expressed as symbols or values

- Identify each token or lexeme as a type of component

- Check that the sequence of token types is legal:
  - satisfies the grammar of the language
  - sequence of tokens forms a complete, legal instruction

Semantic analysis

- e.g.

`start`	`andi`	`a0`	`,`	`a1`	`,`	`Mask`	`;`	`comment`

- “start” and “Mask” are symbols
- How are their values found?
- The value of “start” is the address of the instruction
- Put the value of “start” into the symbol table
- Mask is defined elsewhere, get the value from the symbol table
- replace the symbol Mask with its value

Code generation

- Use the output of semantic and syntactical analysis to generate binary code
- Each component of the instruction maps to a component of the binary code
- The structure of the instruction set enables this mapping to be relatively straightforward. e.g. for:

```
start  andi a0, a1, 0x0F ; Mask, leave the 4 lsbs
```

andi	AL	a0	,	a1	,	#0x0000000F	;	comment
E	20	1		0		00F		

- The instruction E201000F is placed at address “start” whose value is in the symbol table.

Relocatable code

- A program is not normally the only code running on a computer
- Possible in some small dedicated systems (microcontrollers) but increasingly unusual:
- - normally there is an OS and other programs running
- How does the assembler know where the code will be placed in memory?
- …it doesn't. So how does it know the addresses of data, branch targets etc?
- Binary code is normally generated such that it can be located at any address ("relocatable")
- Much easier when addresses used in the code are not absolute
- - absolute addresses would have to be calculated and then placed into the code each time the code was loaded.

Solution: relative addresses

- Addresses specified relative to the code location are relocatable
- RISC-V code implicitly generates relative addressing:
  1. `la` generates PC-relative addresses
  2. Branch instructions are PC-relative
  3. `lw/sw` specify the address of the data as an offset from the PC
- All of these automatically generate relocatable code.

Summary

- Assembly is the process of converting assembly language to binary or executable files.
- Normally a piece of software (an “Assembler”) is used
- An assembler has to make multiple passes through the code to generate a final binary version
- Code normally needs to be relocatable
- An architecture that uses offset addressing generates more readily relocatable code